Project: Authentication & API Gateway

Due: 03/21/2026 11:59 PM
Extended Deadline: 03/22/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Ingress + Gateway
- Task Overview
- Task

KrakenD
- Task Overview
- Task

Django Authentication
- Task Overview
- Task

Securing LLM API
- Task Overview
- Task

Wrap Up
- Conclusion

Ingress + Gateway (Task)

Assumptions

This project assumes the following:

1. You have a working version of the `Productionizing your LLM Server` project. Otherwise, please reach out to the TAs for a working solution.
2. You have set up your Kubernetes cluster, pushed the container image to the ACR, and confirmed that both the `/healthcheck` and `/api?message=hi` endpoints return successful responses. The skills covered in previous projects — connecting your development machine to the cluster, building and pushing your image to the ACR, and deploying it to the cluster — are expected carry-overs here. If you need a refresher on any of these steps, refer back to those projects.

Pre-Project Structure

For the Unit 1 project, we built a FastAPI-based LLM application. In the Unit 0 project, we set up the required infrastructure using Terraform. This project uses both of these components.

You are required to clone your handout repository to your student VM. As a refresher, we expect your directory structure to look like the following:

```
├── README.md
├── answers
│   └── ...
├── llm_server
│   ├── Makefile
│   ├── README.md
```

```
│   ├── docker
│   │   └── Dockerfile
│   ├── k8s
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   ├── llm_server
│   │   ├── __init__.py
│   │   └── main.py
│   ├── poetry.lock
│   ├── poetry.toml
│   ├── pyproject.toml
│   ├── tests
│   │   ├── __init__.py
│   │   └── test_requests.py
│   └── tinyllama-1.1b-chat-v1.0.Q2_K.gguf
├── ruff.toml
├── state.json
├── submitter
├── terraform
│   ├── main.tf
│   ├── terraform.tfstate
│   ├── terraform.tfstate.backup
│   └── variables.tf
├── terraform_init.log
```

Project Architecture

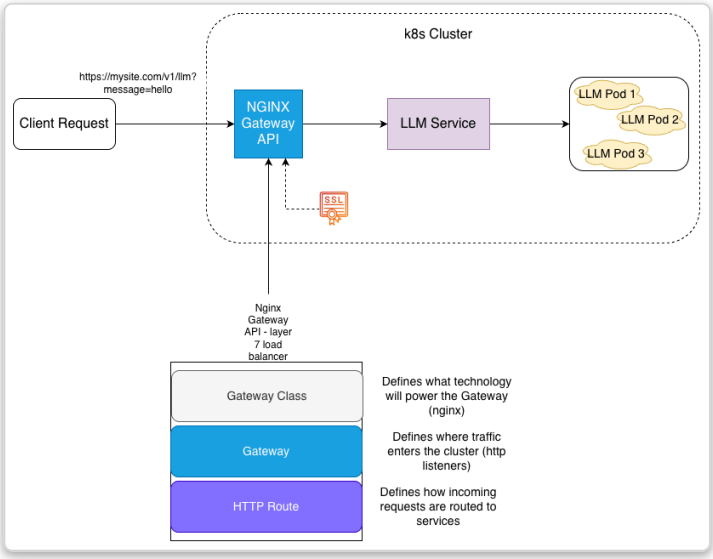

Figure: Task 1 Architecture Diagram

Take some time to review the diagram and familiarize yourself with each component. Understanding how these pieces work together will make it much easier to follow the remaining steps of the project.

Helm Setup

This project makes use of helm, a package manager for Kubernetes. Helm packages, configures, and deploys applications as reusable **charts** that contain templated YAML manifests for resources such as Deployments, Services, and ConfigMaps.

Install Helm if you do not already have it, as it is required for the following steps.

CD into your home directory and run the following commands to install helm:

```
curl -fsSL -o get_helm.sh https://raw.gith
chmod 700 get_helm.sh
./get_helm.sh
```

Load Balancers, Ingresses, and Gateways

This project is designed to teach you four competing ideas:

- The complexities of traffic management for a single service, and what control you have over it
- API management
- Django, including a login service
- OAuth2, so you can understand a more complex form of microservice communication

Each task targets one of these core topics. This task will focus on **traffic management**.

Traffic Management: A Review

In previous projects, we mostly created Kubernetes Services that exposed our microservices to the world. While this is useful for testing, if we want to turn our LLM service into a real product, we need to be more thoughtful about how we expose it to the internet.

Let's review the options Kubernetes provides.

Kubernetes Primitives for Exposing a Service

Kubernetes offers four primary ways to expose a Service:

- **ClusterIP**: This is the default. It creates an internal IP that can be accessed only within the cluster (or locally via `kubectl proxy`). This is generally the

best option for internal communication because it avoids exposing services externally, but it can make certain tasks (like grading) harder.

NodePort: This exposes the service on a static port on every node in the cluster (typically in the range `30000–32767`). This can work well for small setups, but it does not provide load balancing across nodes — you must handle that yourself.

LoadBalancer: This has been our workhorse in previous projects. It provisions an external IP address that you can send requests to. However, Kubernetes does not provide this natively; it relies on the cloud provider (in our case, Azure) to provision a Layer 4 load balancer. As a result, it is more expensive per service and exposes public IP addresses, but it supports common protocols and is easy to use.

ExternalName: This does not expose a service directly. Instead, it maps a Kubernetes service name to an existing DNS name.

It is important to note that none of these approaches scale cleanly when you have many microservices. They are also primarily designed for internal (east–west) traffic rather than external (north–south) traffic. What we want is an approach that scales to multiple services and provides more configuration options so we can better control how users access our system.

Ingresses and the Gateway API

> **Warning**
>
> As mentioned in lecture, there is an important difference between “API Gateways” and the Kubernetes “Gateway API.” This task focuses on the latter; the next task will focus on the former.

A **Kubernetes Ingress** is an API object that manages external HTTP/HTTPS access to services inside a cluster by defining routing rules. One common example is **NGINX Ingress**, which routes external traffic to internal services. Ingress is the standard Kubernetes approach for exposing HTTP/S services using **Layer 7 routing** (as opposed to Layer 4 routing provided by most cloud load balancers).

Layer 7 routing lets you route traffic based on parts of the HTTP request, not just IP/port. This enables features

such as:

- Routing based on paths (e.g., `/hi` vs. `/llm`), either exact matches or prefixes
- Basic protections against large payloads and other error cases
- Routing based on the `Host` header (virtual hosting)

Ingress also enables other important capabilities, including:

- **TLS termination**, so individual microservices do not need to manage HTTPS themselves
- **URL rewriting**, so you can maintain older URLs while migrating to new APIs

While Ingress works well in many cases, it has some limitations:

- **Lack of standardization**: Advanced configuration is often controller-specific. If you need full customization, you must learn your controller's annotations and behavior, and you may need to operate the controller like another service.
- **Limited authorization boundaries**: In real-world clusters, platform teams often want stricter control over who can expose services publicly and how. The Gateway API provides clearer separation between cluster operators and application teams.
- **Limited protocol support**: Ingress is primarily designed for HTTP/S. While HTTP/S is powerful, it is not the only protocol you may need.
- **Limited independent management**: If multiple teams share an Ingress resource, they must coordinate changes to avoid conflicts. The Gateway API improves this by allowing teams to manage routes independently while sharing the same gateway.

Because of these limitations, Kubernetes introduced the **Gateway API** as a newer, more standardized approach. It aims to unify the Ingress ecosystem, improve portability across environments, and provide a cleaner separation of responsibilities between infrastructure providers, cluster operators, and application developers.

In this task, we will take our LLM service deployment and enable Azure's Gateway API controller: **Azure Application Gateway**.

Information

In Azure, there are two ways to do this. You can provision the ALB separately using the Azure CLI (what Azure calls “BYO deployment”), or you can manage it through Kubernetes resources. For this project, we will use the Kubernetes-based approach, since external resources are easier to forget during cleanup.

The counterpart to NGINX Ingress for the Gateway API is **NGINX Gateway Fabric**, which we will use for this task.

To install NGINX Gateway Fabric, we first need to install the Gateway API **Custom Resource Definitions (CRDs)** on our cluster.

A *CRD* is a way to extend the Kubernetes API with your own resource types.
By default, Kubernetes comes with built-in resource types like Pod, Deployment, and Service. A CRD lets you define a new kind of resource that Kubernetes doesn't know about out of the box — once installed, you can create, manage, and interact with it just like any native Kubernetes object (via kubectl, YAML manifests, etc.).

In the context of the Gateway API, the CRDs define new resource types like `Gateway` and `HTTPRoute` that don't ship with Kubernetes by default. Installing the CRDs is essentially telling your cluster "*hey, these new resource types exist*" — after that, you can start creating them.

These CRDs are published by the official Kubernetes Gateway API project. Run the following command to install the CRDs:

```
kubectl apply -f https://github.com/kubern
```

Next, install the `nginx-gateway-fabric` Helm chart, which deploys all required components:

```
helm install ngf oci://ghcr.io/nginxinc/ch
  --namespace nginx-gateway \
  --create-namespace
```

Wait for the deployment to become available:

```
kubectl wait --timeout=5m -n nginx-gateway
```

You can confirm the installation by listing the available `GatewayClass` resources:

```
kubectl get gatewayclass -o yaml
```

At this point, the `GatewayClass` is installed. In other words, Kubernetes now knows about this gateway implementation, and you can create Gateways of this type in the next steps.

Subtask 1: Create a Gateway Resource

In the project root folder, create a directory named `k8s-components` and `cd` into it. Your first task is to create a file called `gateway.yaml`.

This tutorial in the official Gateway API docs is the best starting point for understanding what `gateway.yaml` should look like: https://gateway-api.sigs.k8s.io/guides/getting-started/simple-gateway/

Use the provided example as a starting point, and update `gatewayClassName` to match the name of your `GatewayClass`. To find the correct name, carefully review the output of:

```
kubectl get gatewayclass -o yaml
```

Make sure your Gateway includes a `listeners` section with an entry that listens for HTTP traffic on port **80**. Once that is completed, run:

```
kubectl apply -f gateway.yaml
```

To verify that the Gateway was created successfully, run:

```
kubectl get gateway
```

Subtask 2: Create Your First HTTPRoute

With the Gateway created, you now need to create an `HTTPRoute` to connect the Gateway to your Service.

Before creating the `HTTPRoute`, update the Service type for your LLM app from `LoadBalancer` to `ClusterIP`. From this point onward, all external traffic should flow

through the Gateway, so you no longer need a public IP on the Service.

To do this, go to `llm_server/k8s`, update `service.yaml` by setting `type: ClusterIP`, and re-apply the manifest. Then list the Service to confirm the change.

Next, refer to the Gateway API docs again to understand how to define an `HTTPRoute`. Create a file called `httproute.yaml`. Use the example from the documentation as a starting point, then:

- Update `backendRefs` to point to the LLM app's Service name and port
- Ensure `parentRefs` points to your Gateway
- Finally, deploy the route by running: `kubectl apply -f httproute.yaml`

Interim Validation

Run the following commands to validate and debug your setup:

```
# List the Gateway
kubectl get gateway

# List the HTTPRoutes
kubectl get httproutes

# Curl to check if the llm-app is reachabl
curl http://<GATEWAY_IP>/api?message=hi

# View events and debugging information fo
kubectl describe gateway
```

Subtask 3: Securing and Productionizing Your Gateway

Now, let's assign a domain name to your public IP address and enable HTTPS.

As a reminder, HTTPS provides transport security. It ensures that network traffic cannot be easily read or modified in transit. To enable HTTPS, we need an **SSL/TLS certificate**. TLS (commonly referred to as SSL) is the security protocol behind HTTPS, and it helps clients (like your browser) verify that they are connecting to the intended server.

There are two common ways to obtain a certificate:

Self-signed certificate: You generate and sign your own certificate. This can work for internal or testing use (where you control both client and server), but browsers typically do not trust self-signed certificates for public websites.

Certificate Authority (CA): You obtain a certificate from a trusted CA. Browsers trust a set of well-known CAs, which allows HTTPS to work without warnings. Most CA-based solutions require you to own a domain name.

Getting a Domain Name

A domain name is a human-readable version of an IP address. Domains are typically purchased through registrars, and prices can vary. For this course, we will use a free external provider that gives you a subdomain.

1. Go to **noip.com** and create an account.
2. Create a free subdomain under **DDNS & Remote Access** → **DNS Records**.
3. Set the DNS record to point to the **Gateway public IP** you used in the previous subtask, so traffic to your domain is routed correctly.

Interim Validation

After your DNS record is set up, verify that the domain works using either `curl`:

```
curl http://YOUR_DOMAIN_NAME/api?message=h
```

Or open the domain in your browser and test the endpoint there.

At this point, you should have a working domain name pointing to your Gateway.

SSL Certificates

Next, we will configure our system to use the domain name with HTTPS. While No-IP does offer paid SSL certificates, the cost is relatively high for what you get—especially since you can obtain a trusted SSL certificate for free from **Let's Encrypt**.

Let's Encrypt is a free, automated, and open certificate authority (CA) run for the public's benefit. It was developed by Alex Halderman, a well-known security

researcher currently focused on the security of voting machines.

Using Let's Encrypt in a Kubernetes cluster is fairly straightforward. The first step is to install **cert-manager**, a Kubernetes add-on that simplifies certificate management.

Add the Helm repository that provides cert-manager charts, update the repository, and install cert-manager:

```
# Add the repo that provides helm charts f
helm repo add jetstack https://charts.jets

# Get the latest chart versions
helm repo update

# Install cert-manager (current version: v
helm install cert-manager jetstack/cert-ma
  --namespace cert-manager \
  --create-namespace \
  --version v1.19.2 \
  --set crds.enabled=true \
  --set config.apiVersion="controller.conf
  --set config.kind="ControllerConfigurati
  --set config.enableGatewayAPI=true
```

In the final command, notice that we explicitly enable Gateway API support. Cert-manager is a general-purpose tool that works with many environments (bare-metal VMs, on-prem clusters, Ingress, etc.), so we must configure it to install the resources required for Gateway API integration.

Creating a ClusterIssuer

Next, create a `clusterissuer.yaml` file inside the `k8s-components` directory. A **ClusterIssuer** is a cluster-wide cert-manager resource that defines how certificates are issued (for example, via ACME/Let's Encrypt) and can be used across all namespaces.

Below is an example `ClusterIssuer` YAML, and you must go through the official documentation to understand what each field is responsible for - here:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
```

```
    name: letsencrypt-staging
  spec:
    acme:
      email: user@example.com
      server: https://acme-staging-v02.api.l
      privateKeySecretRef:
        name: issuer-account-key
      solvers:
      - http01:
          ingress:
            ingressClassName: nginx
```

You will need to modify this example to work with your Gateway-based setup. Specifically:

- Update the `email` field to your own email address.
- Change the `server` value to the **production** Let's Encrypt endpoint: `https://acme-v02.api.letsencrypt.org/directory`
  The staging server is useful for testing, but only the production server issues certificates trusted by browsers and `curl`.
- Update the `solvers` section so cert-manager knows how to complete HTTP-01 challenges using your **Gateway**. Refer to the cert-manager Gateway API documentation for an example, and make sure you correctly specify the Gateway name and namespace.

Interim Validation

To verify that your ClusterIssuer is registered correctly, run:

```
kubectl describe clusterissuer
```

At the end of the output, you should see something similar to:

```
    Conditions:
      Last Transition Time:  2026-03-01T01:4
      Message:               The ACME accoun
      Observed Generation:   1
      Reason:                ACMEAccountRegi
      Status:                True
```

```
    Type:                    Ready
  Events:                    <none>
```

Updating the Gateway and Routes for HTTPS

Once the ClusterIssuer is ready, update your existing Gateway and HTTPRoute resources to enable HTTPS.

Rather than deleting and recreating resources, edit the existing YAML files and reapply them using `kubectl apply`. This follows Kubernetes' declarative deployment model and is considered best practice.

Your goal is to enable **HTTPS access** to your Gateway and eventually disable **HTTP access** to your LLM service.

Start by updating the Gateway YAML:

Under `metadata.annotations`, add `cert-manager.io/cluster-issuer` and set it to the name of your ClusterIssuer.
Under `listeners`, add a `hostname` field. This should be your domain name only (no protocol or port), for example: `project3cloudnative.viewdns.net`.
Add an **HTTPS listener** on port 443. Configure TLS termination so the Gateway handles decryption instead of your pods. You can use the following example as a reference:

```
- name: https-listener
  port: 443
  protocol: HTTPS
  allowedRoutes:
    namespaces:
      from: Same
  tls:
    mode: Terminate
    certificateRefs:
    - kind: Secret
      group: ""
      name: listener-tls-secret
```

Do **not** remove the HTTP listener yet. Cert-manager requires HTTP access to verify domain ownership. You can monitor this process using `kubectl get orders` and `kubectl get certificates`. The certificate

order must reach a `valid` state. Once verification is complete, you may remove the HTTP listener to disable plain HTTP access.

After updating the Gateway, running `kubectl describe gateway` should show events similar to:

```
Events:
  Type     Reason             Age
  ----     ------             ----
  Warning  BadConfig          9s (x4 over
  Normal   CreateCertificate  9s
```

The ClusterIssuer will create a Kubernetes Secret containing the TLS certificate. This process may take 2–3 minutes.

Next, update your HTTPRoute:

In `parentRefs`, set `sectionName` to the listener you want the route to attach to.
Under `spec`, add the `hostnames` field and specify your domain name.

Debugging

To debug issues with certificate setup—and when posting on Ed Discussion—include the output of the following commands. You will almost certainly find logs here that point you to the right next debugging step.

```
kubectl get gateway

# If this doesn't return anything, it mean
kubectl get certificates

# Similar to certificates: if this doesn't
# If this is stuck in Pending, make sure y
kubectl get orders

# These often include useful failure event
kubectl describe certificates

# Same
kubectl describe orders
```

Final Validation

Once everything is configured, validate your setup by accessing:

- `https://MY_DOMAIN_NAME/api?message=hi` in your browser, or
- Using `curl` with HTTPS.

Cluster State

At the end of this task, your cluster state should look something like this:

```
azureuser@qa-vm:~/cloud-native-handout-mik
NAME              TYPE        CLUSTER-IP    
kubernetes        ClusterIP   10.0.0.1      
llmapp-service    ClusterIP   10.0.30.76    

azureuser@qa-vm:~/cloud-native-handout-mik
NAME             CLASS   ADDRESS           P
nginx-gateway    nginx   20.127.176.146    T

azureuser@qa-vm:~/cloud-native-handout-mik
NAME        HOSTNAMES                      
llmroute    ["s26cloudnative.myvnc.com"]
```

Directory Tree Structure

Your directory tree structure should look like below:

```
k8s-components/
├── clusterissuer.yaml
├── gateway.yaml
└── httproute.yaml

1 directory, 3 files
```

Submission Information

As long as you can successfully connect to your cluster using the configured hostname, you can submit your work using:

```
./submitter -t task1
```

Information

Note: This submitter uses `kubectl` during grading for this and all subsequent tasks.

Please ensure `kubectl` is installed, available in your PATH, and `kubectl get pods` works without requiring a custom path.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform